



Sistemas Distribuidos I (75.74)

Arquitecturas Cloud

Cloud Computing. Platform as a Service (PaaS). AppEngine.
BigTable.

Docentes

- | | | |
|---------------------|-----------------------------|-------------------|
| ■ Pablo Roca | ■ Manuel Reberendo | ■ W. Gabriel Diem |
| ■ Franco Barreneche | ■ Máximo Gismondi | ■ Máximo Utrera |
| ■ Nicolás Zulaica | ■ Camila Sebellin | ■ Ivan Pfaab |
| ■ Franco Papa | ■ Patricio Tourne Passarino | |

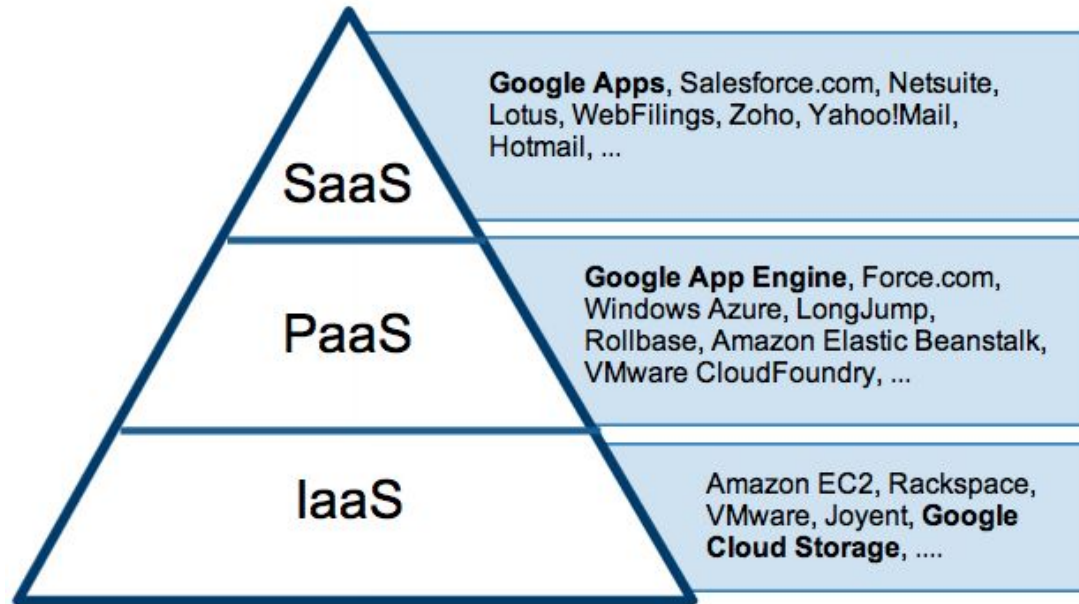


● Cloud Computing

- Platform as a Service (PaaS)
- Caso de estudio: Google AppEngine y Datastore
- BigTable



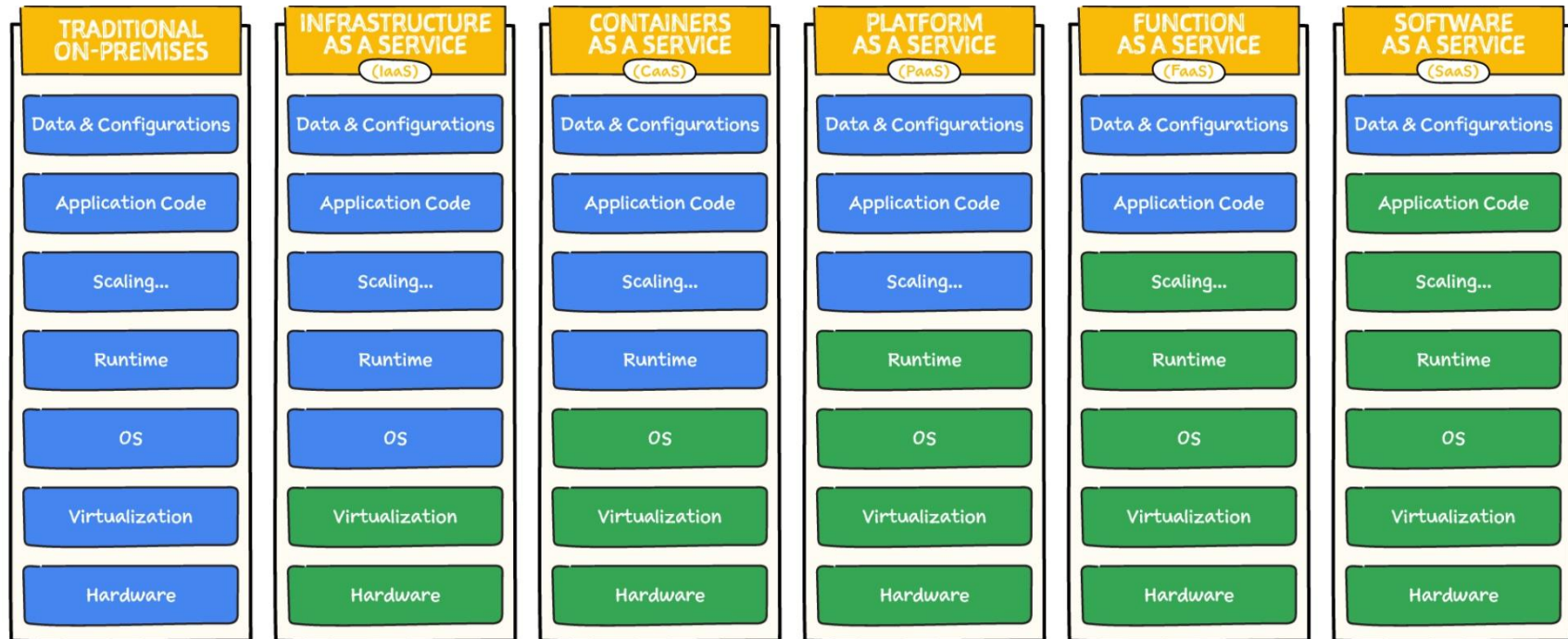
- Metáfora para internet y todo el contenido que ofrece.
- “Todo lo que se pueda consumir más allá del firewall...”
- Una forma de ofrecer recursos de IT, no necesariamente una nueva tecnología.
- Networking + Infraestructura + Nuevas Plataformas + Servicios



Source: Gartner AADI Summit Dec 2009



- **Infrastructure as a Service (IaaS)**
 - Almacenamiento y virtualización de equipos.
 - Definir redes y técnicas de adaptación para capacidad frente a cargas.
- **Platform as a Service (PaaS)**
 - Frameworks y plataformas para desarrollar aplicaciones *Cloud ready*.
 - Recursos expuestos como servicios para el desarrollo y el manejo del ciclo de vida de las aplicaciones (logs, *monitoring*, etc).
- **Software as a Service (SaaS)**
 - Software a demanda. Alquiler de servicios.
 - Ámplia variedad de soluciones genéricas y adaptables.
 - Protocolos abiertos y arquitecturas pensadas para integración.



<https://cloud.google.com/learn/paas-vs-iaas-vs-saas>

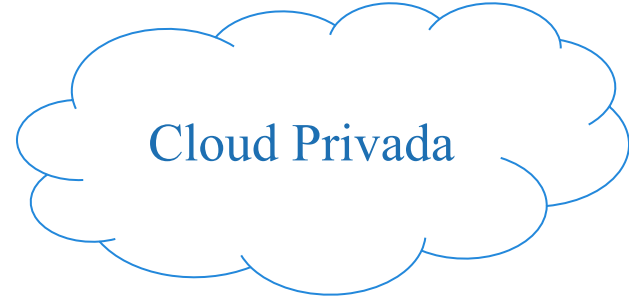


Cloud | Principales Beneficios

- **Accesibilidad:** *Access anywhere.* Movilidad y visibilidad constante de los recursos.
- **Time-to-Market:** Disponibilidad instantánea de los recursos.
- **Escalabilidad:** Capacidades 'ilimitadas' de recursos para manejar volúmenes: almacenamiento, ancho de banda, cómputo, memoria, etc.
- **Costos:**
 - Pago a demanda (*Pay as you go*)
 - Control del gasto dependiendo del uso
 - Accesibilidad + escalabilidad + confiabilidad realmente barata



- Servicios públicos
- Servidores compartidos con otros usuarios
- Disponibilidad de recursos garantizados con SLAs
- Costos variables. *'Pay as you go'*
- Se accede mediante internet



- Servicios privados
- Datacenter propio de la empresa
- Recursos dedicados
- Costos fijos de mantenimiento y expansión
- Se accede mediante intranet



- Factores Políticos
 - Licenciamiento, jurisdicción y pérdida de gobernabilidad sobre los datos
 - Incapacidad de influir sobre la toma de decisiones que afecta al hardware
- Factores Técnicos
 - Costos monetarios y de tiempo para migraciones de datos y software
 - Alta exposición de datos sensibles
 - Adaptación de los sistemas a los nuevos modelos de arquitectura cloud

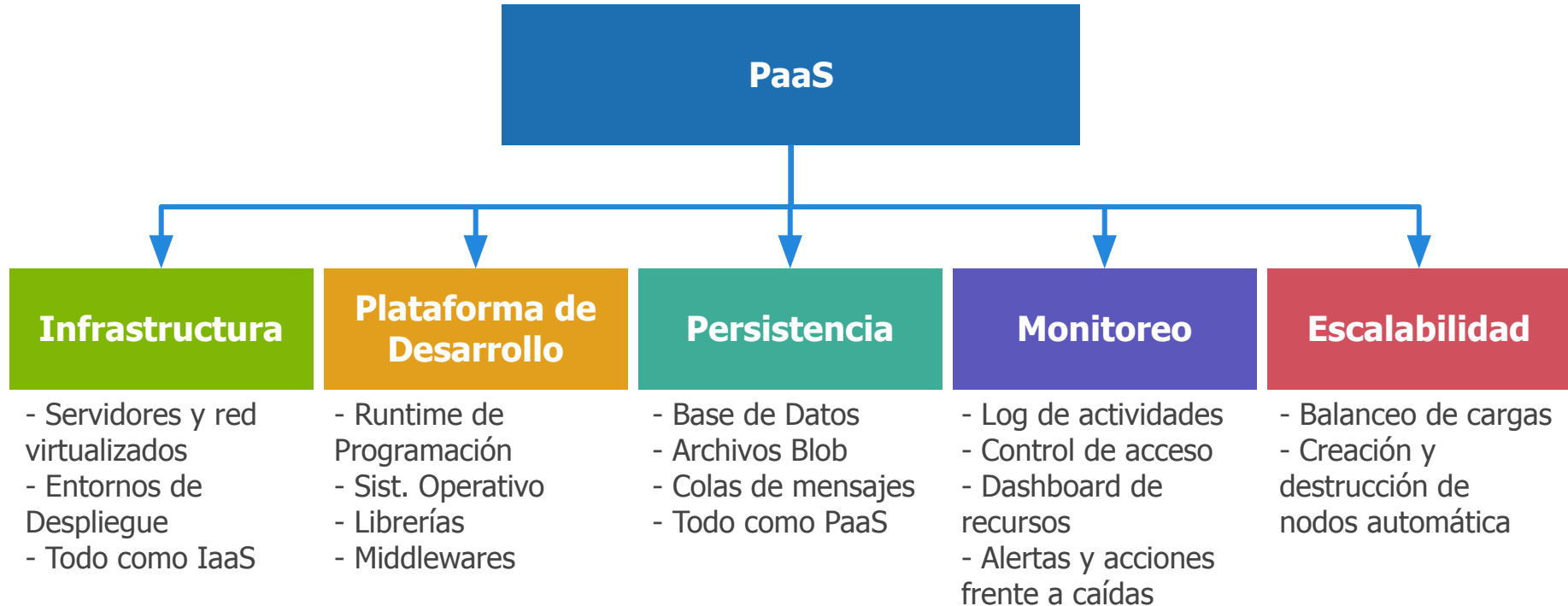
Agenda



- Cloud Computing
- **Platform as a Service (PaaS)**
- Caso de estudio: Google AppEngine y Datastore
- BigTable



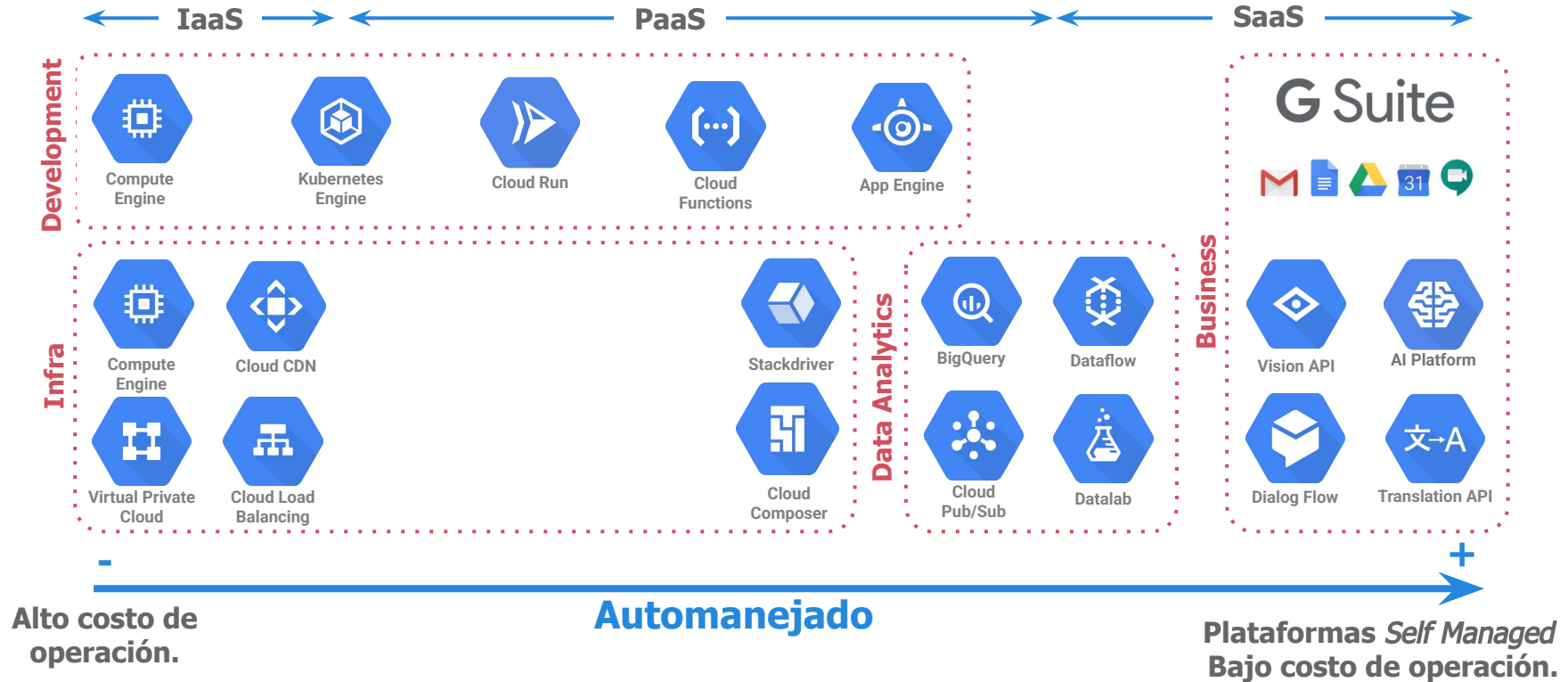
Platform as a Service (PaaS)



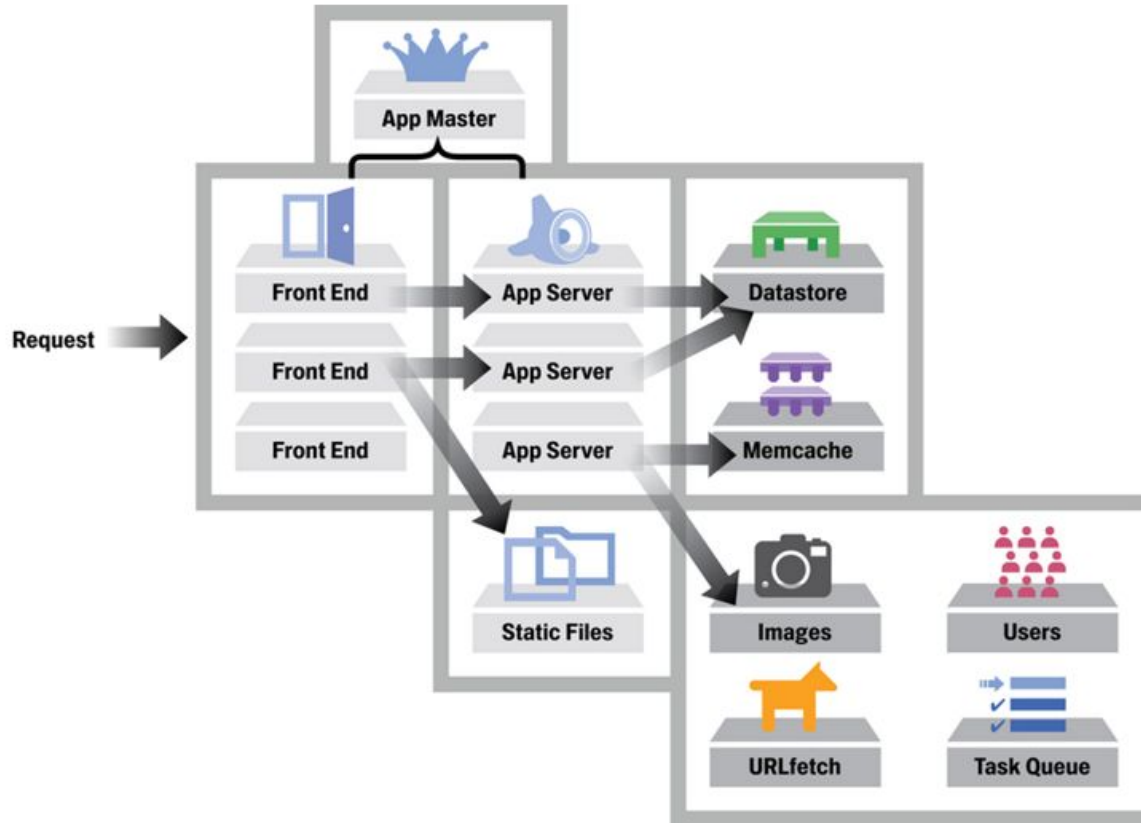
Google Ecosystem



Target Audience



Infraestructuras Escalables | Modelo AppEngine



Agenda



- ☐ Cloud Computing
- ☐ Platform as a Service (PaaS)
- ☒ **Caso de estudio: Google AppEngine y Datastore**
- ☐ BigTable

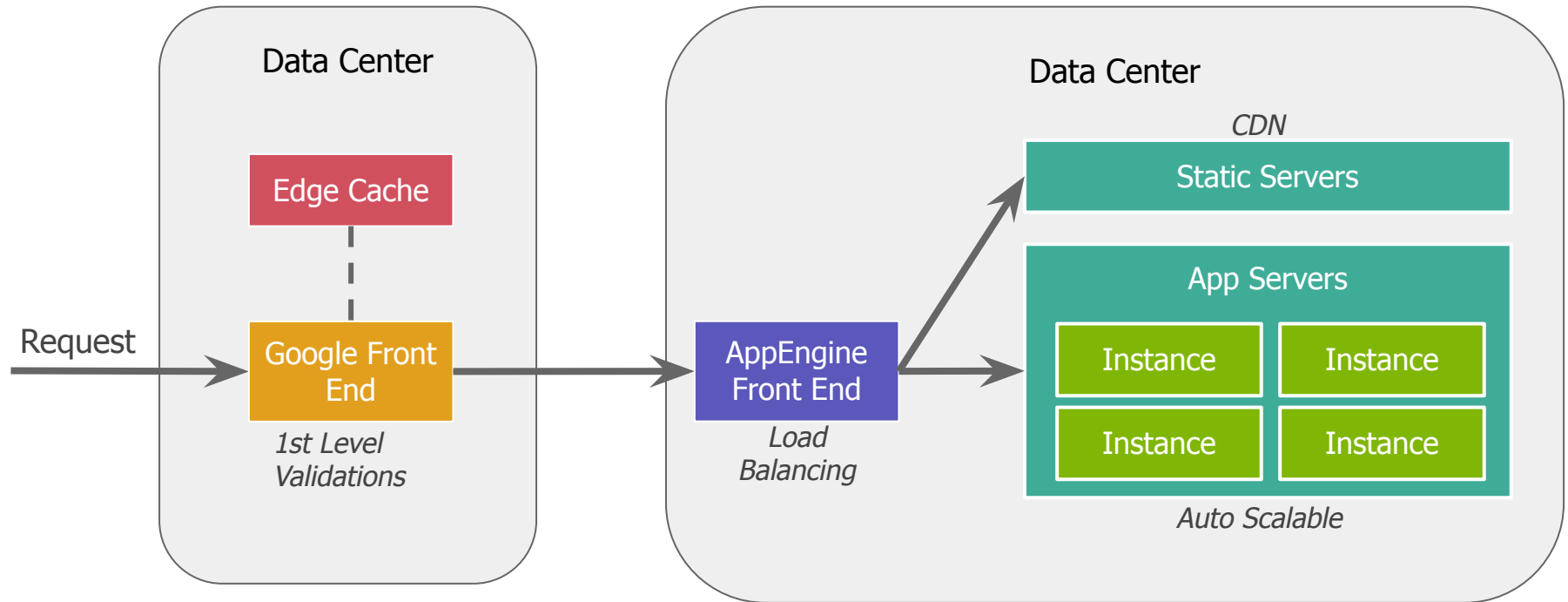


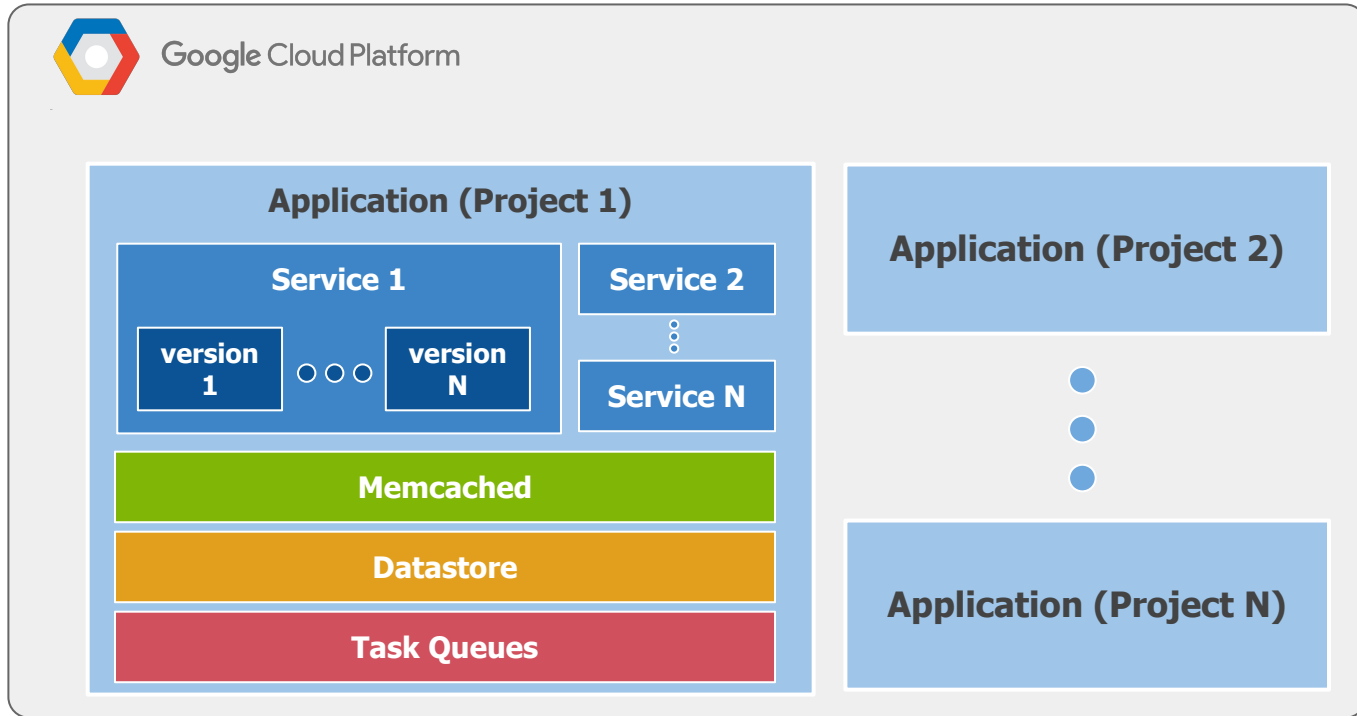
Google AppEngine | Objetivos de Diseño

- Las buenas prácticas son forzadas
 - Sistemas granulares
 - Escalamiento horizontal
 - Requests breves con posibilidad de encolar Requests largos
 - Independencia del SO y Hardware
- Servicios de aplicación ya integrados:
 - Cache
 - Colas de mensajes
 - Elasticidad
 - Versionado
 - Herramientas de Log, *Debugging* y Monitoreo
 - Modelos No-Relacionales con Datastore/BigTable



AppEngine







- Servicios = Módulos
 - Permite mantener unidad entre las operaciones soportadas.
 - Pueden desplegarse distintas versiones
 - Para cada versión puede existir una o más instancias
- Instancias = AppServers = Backend Servers
 - Son la unidad de procesamiento
 - Se las clasifica en Dinámicas o Residentes
 - Dinámicas: Son creadas por los requests
 - Residentes: Escaladas de forma manual



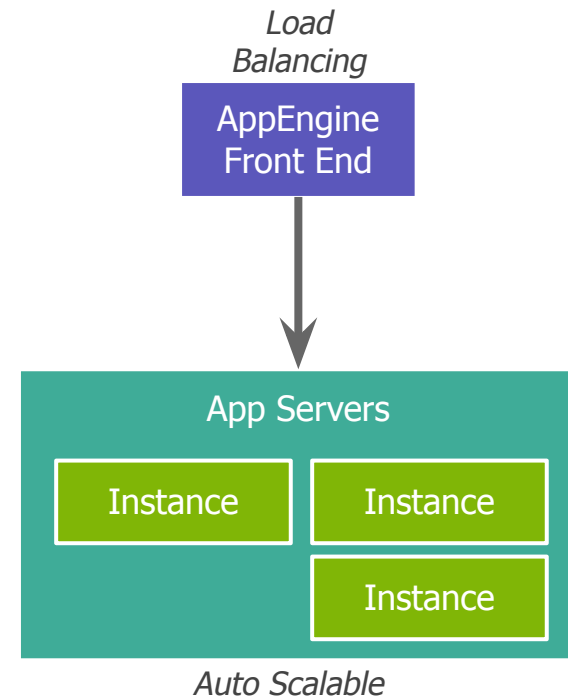
Google AppEngine | Instancias de Procesamiento

Instancias Dinámicas

- Se crean dinámicamente
- Procesan request pequeños
- Fuerzan respuestas rápida y manejo sin estado (stateless)
- Pueden aceptar requests externos (GET) e internos (mensajes en colas 'push')

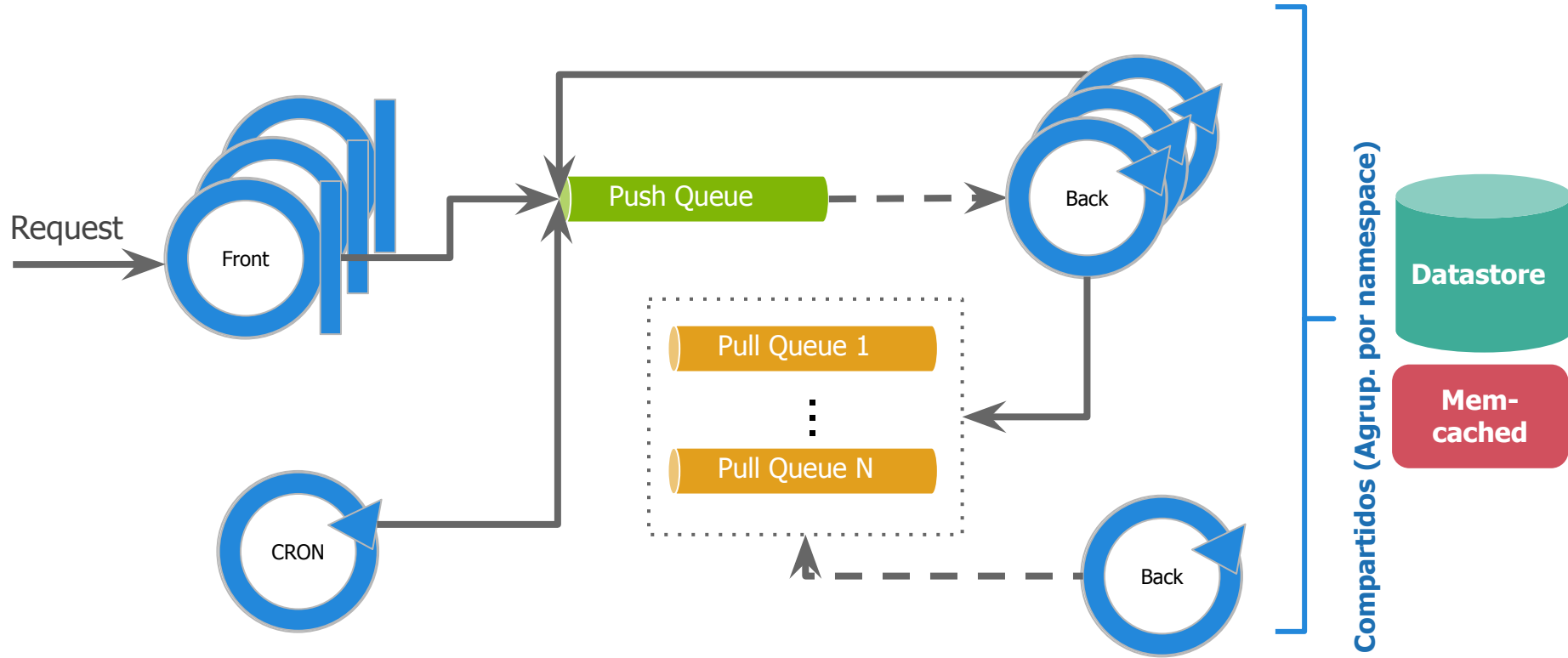
Instancias Residentes

- Creadas de forma manual mediante configuración.
- No existen límites para su empleo y se pueden elegir su capacidad de cómputo
- Procesan requests largos, especialmente en *batches* con o sin estado.





Google AppEngine | Comunicación Interna





Push Queue



- Nombre
- URL
- Headers

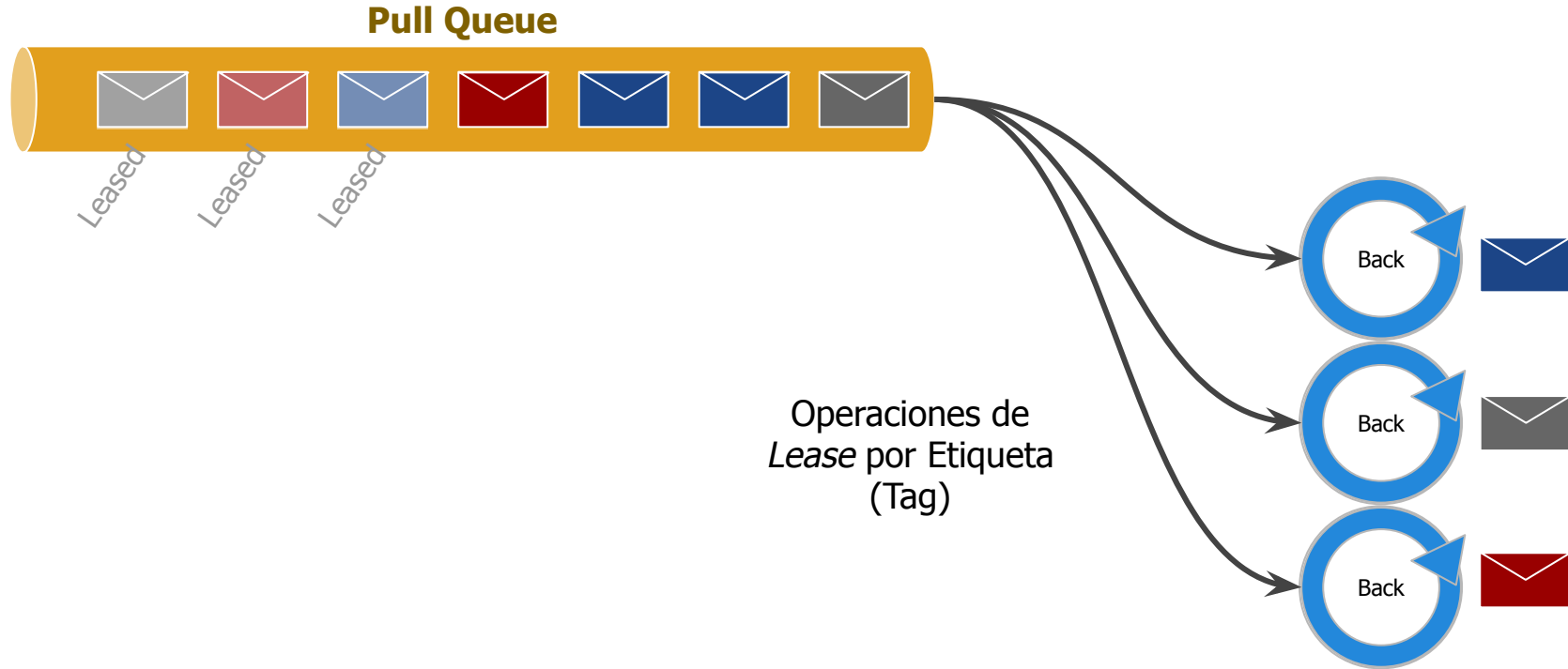
- Envían los requests a instancias activas
- La URL define la instancia, servicio y versión p/atender el mensaje
- El payload está dado por los argumentos en la URL y headers

Pull Queue



- Nombre
- *Payload*
- Etiqueta

- Permiten encolar tareas a ser consumidas de forma controlada.
- Es el enfoque más usual de colas de tareas y se puede reemplazar por colas PubSub





Google AppEngine | Almacenamiento

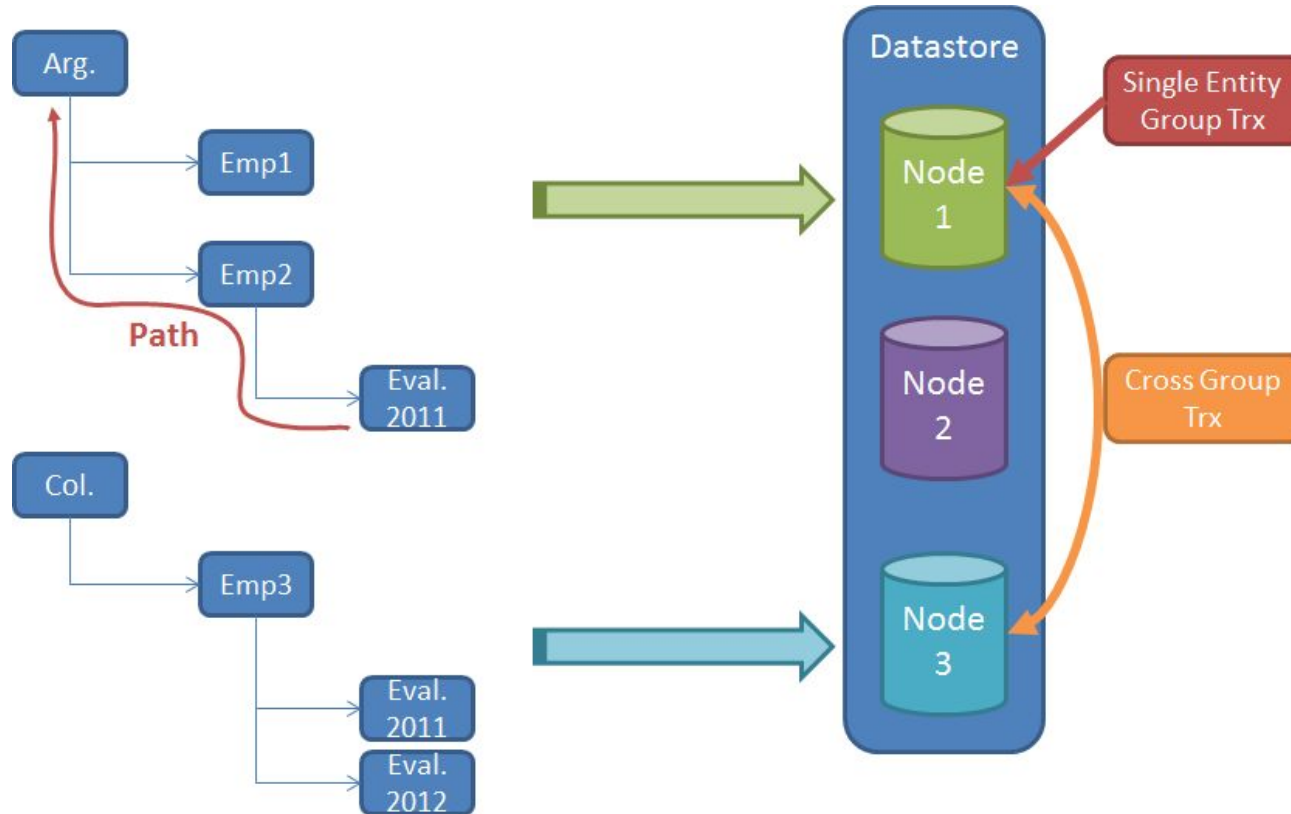
- Base no-SQL administrada por Google y accesible en modalidad PaaS
 - Modelo de objetos con atributos: *Entities*
 - Tipos sin esquema con atributos indexables: *Kinds*
- Altamente escalable (volumen de datos)
 - Basada en BigTable (que es soportada por un Google FS: Colossus)
 - Soporta entidades de hasta 1MB
 - Soporta consultas por Key o por atributos indexados.
 - Millones de escrituras por segundo. No es bueno para consultas.
 - Operaciones ACID incluso entre entidades.



Datastore



Google AppEngine | Particionamiento del Almacenamiento



Agenda



- Cloud Computing
- Platform as a Service (PaaS)
- Caso de estudio: Google AppEngine y Datastore
- **BigTable**



Claves, datos y columnas

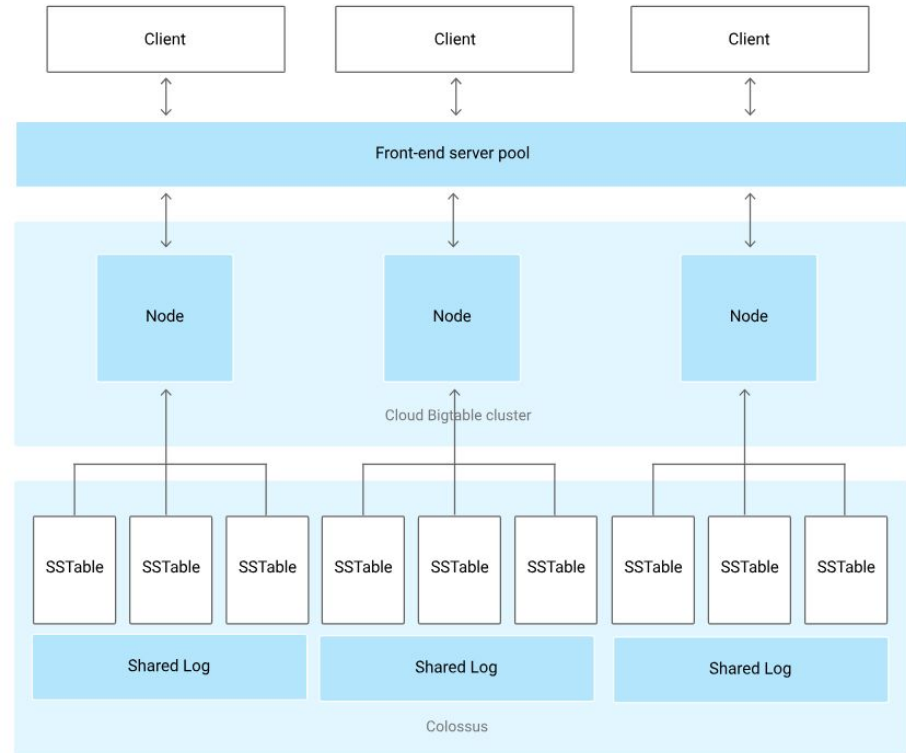
- Solo almacena pares clave-datos.
- Los datos son en realidad un conjunto de valores (o columnas).
- Al tratarse de conjuntos dispersos (*sparse*), los valores no se almacenan en un orden definido sino que conocen su 'column family'.

	Column family 1		Column family 2	
	Column 1	Column 2	Column 1	Column 2
Row key 1				
Row key 2				



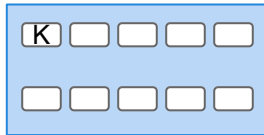
Tablets

- Conjunto de filas consecutivas de acuerdo a la clave.
- Unidad de balanceo de BigTable. Permite escalar el sistema.

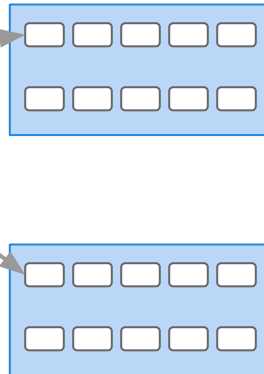




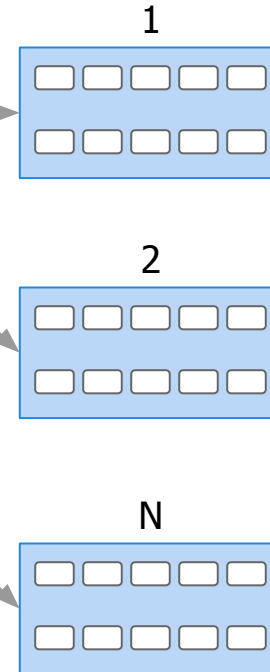
**Root Metadata
(1st level tablets)**



(2nd level tablets)



**User Level Table
(3er level tablets)**



Solo 3 niveles. Similar a Árboles B +.

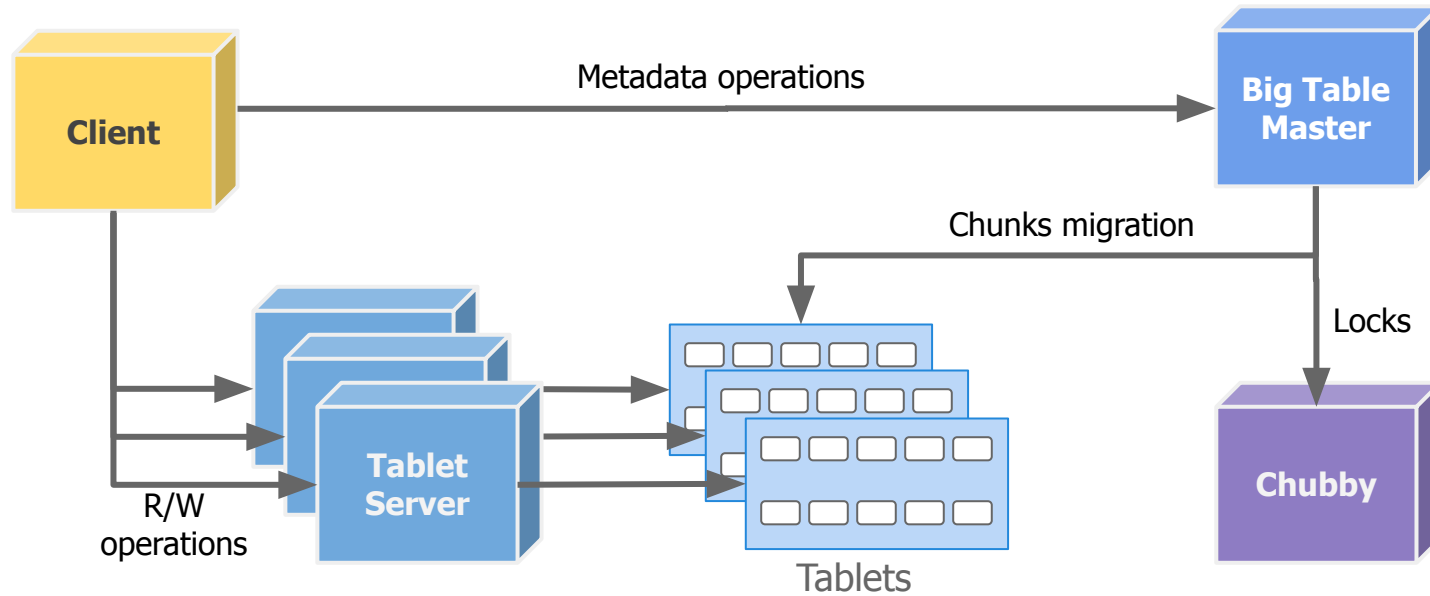


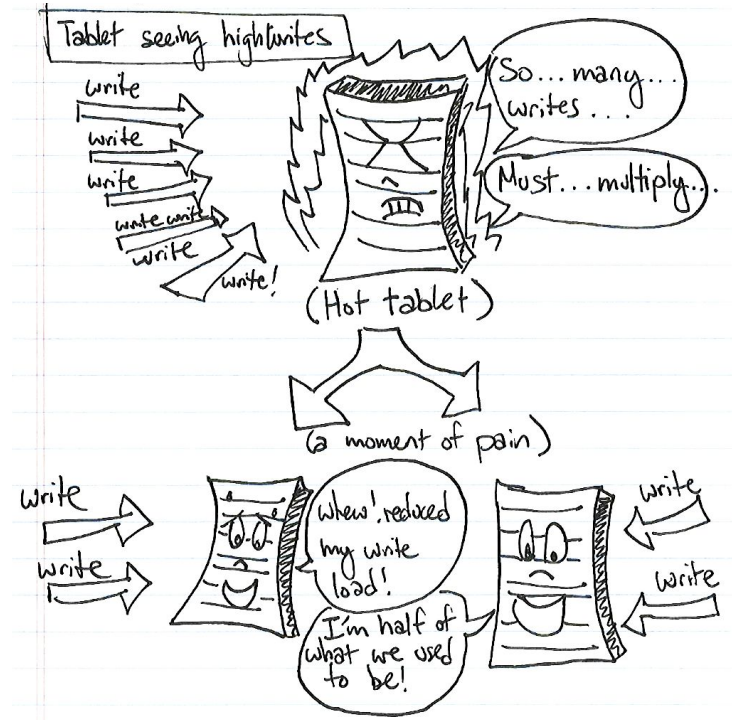
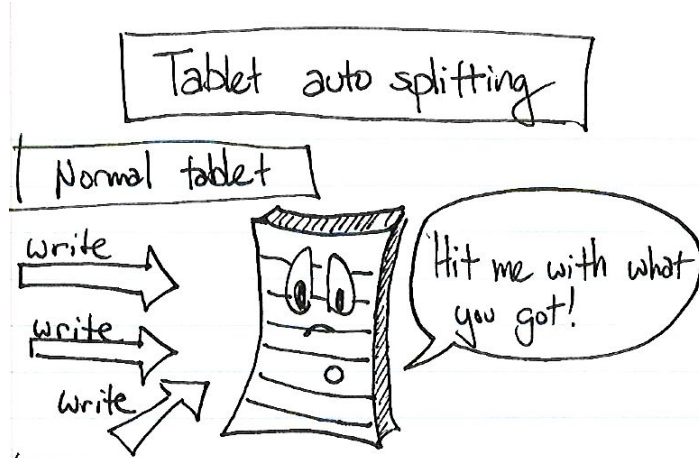
Ventajas

- Operaciones atómicas por clave
- Lecturas por rangos de claves -> Principio de Localidad de datos
- Versionado de celdas

Restricciones

- Imposibilidad de crear índices:
 - Organizar indexado en base a la clave
 - Simulación de índices con duplicación de datos en nuevas tablas
- Imposibilidad de crear contextos de transacción *cross-keys*





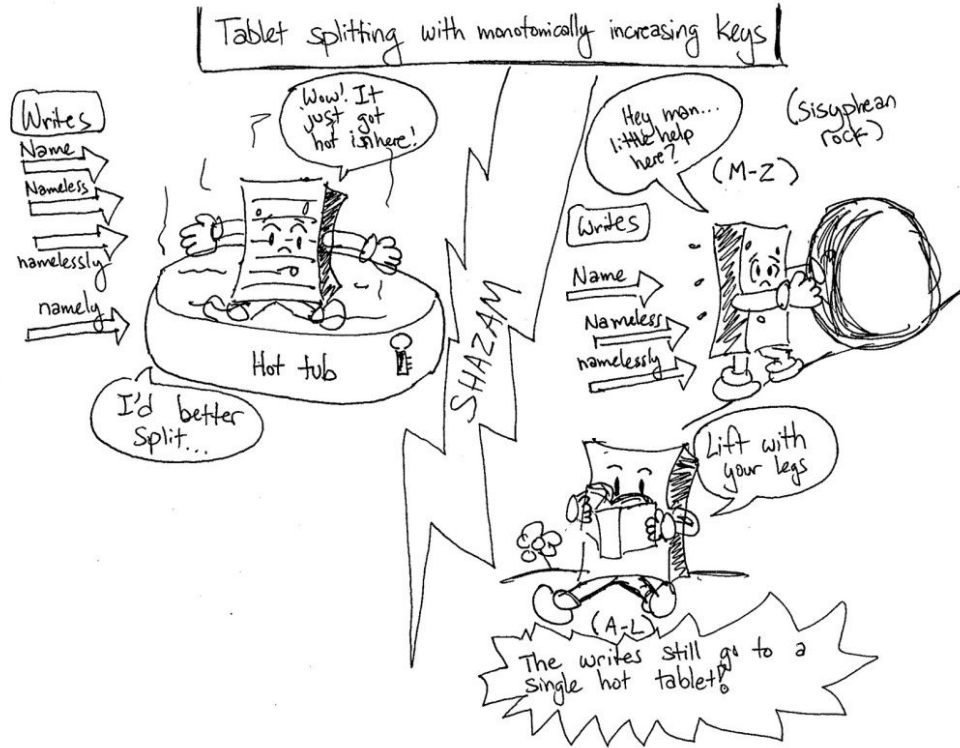
Source:
<https://ikaisays.com/2011/01/25/app-engine-datastore-tip-monotonically-increasing-values-are-bad/>, Ikai Lan

BigTable | División de Tablets Exitoso



Source:
<https://ikaisays.com/2011/01/25/app-engine-datastore-tip-monotonically-increasing-values-are-bad/>, Ikai Lan

BigTable | División de Tablets No Exitoso



Source:
<https://ikaisays.com/2011/01/25/app-engine-database-tip-monotonically-increasing-values-are-bad/>, Ikai Lan



Bibliografía

- Google I/O 2011: Scaling App Engine Applications, Justin Haugh, Guido van Rossum, 2011
 - <https://www.youtube.com/watch?v=rP-kjrx9CRE&t=1589s>
- Writing Infinitely Scalable and High Performance Apps with AppEngine, Karan Goel, 2017
 - <https://www.youtube.com/watch?v=WpWPMMC0q3E>
- Google Cloud Datastore Inside-out, Etsuji Nakai, 2017:
 - <https://www.slideshare.net/enakai/google-cloud-datastore-insideout>
- App Engine datastore tip: monotonically increasing values are bad, Ikay Lan, 2011:
 - <https://ikaisays.com/2011/01/25/app-engine-datastore-tip-monotonically-increasing-values-are-bad/>
- Ejemplos - Terraform para despliegue en Cloud:
 - <https://github.com/7574-sistemas-distribuidos/terraform-example>
- BigTable, referencia oficial:
 - <https://cloud.google.com/bigtable?hl=es>
- Sami Zuhuruddin, BigTable in Action, 2017.
 - <https://www.youtube.com/watch?v=KaRbKdMInuc>
- Ejemplos - BigTable
 - <https://github.com/7574-sistemas-distribuidos/bigtable-example>